Step 5 — Training: aggregatore di evidenza, prima congelato poi congiunto

Obiettivo

Addestrare la testa di evidenza sopra il modello del verdetto, senza peggiorarlo e senza aggiungere costo in inferenza. Due fasi: **testa su backbone congelato** (il verdetto è identico per costruzione), poi, solo se serve, **training congiunto** LoRA + testa.

Prerequisito

Un modello del verdetto addestrato con il piano binario **usando il claim prima del documento** (02d). Se non c'è ancora, la fase 1 si può provare sul modello di partenza (stesso ordine), sapendo che andrà rifatta sul modello fine-tunato.

1. L'architettura della testa

```python
class EvidenceHead(nn.Module):
    def __init__(self, d_model, d=256, layers=2, heads=4, max_sent=512, n_layers_in=1):
        super().__init__()
        self.proj_h = nn.Linear(d_model * n_layers_in, d)   # stati di fine frase
        self.proj_q = nn.Linear(d_model * n_layers_in, d)   # stato in "Answer:"
        self.pos = nn.Embedding(max_sent, d)                # indice della frase
        self.type = nn.Embedding(2, d)                      # 0 = q, 1 = frase
        enc = nn.TransformerEncoderLayer(d, heads, 4 * d, dropout=0.1, batch_first=True, norm_first=True)
        self.encoder = nn.TransformerEncoder(enc, layers)   # attenzione PIENA (non causale)
        self.out = nn.Linear(d, 1)

    def forward(self, q, h, sent_mask):
        """q: [B, D]; h: [B, N, D]; sent_mask: [B, N] (1 = frase reale) -> logit [B, N]."""
        idx = torch.arange(h.size(1), device=h.device)
        x_h = self.proj_h(h) + self.pos(idx) + self.type.weight[1]
        x_q = (self.proj_q(q) + self.type.weight[0])[:, None]
        x = torch.cat([x_q, x_h], 1)
        pad = torch.cat([torch.zeros_like(sent_mask[:, :1]), 1 - sent_mask], 1).bool()
        return self.out(self.encoder(x, src_key_padding_mask=pad))[:, 1:, 0]
```

Componente	Valore	Perché
Ingresso	stati di fine frase `h_k` e stato finale `q`	gli unici due tipi di stato utili, già calcolati dal forward
Layer letto	ultimo layer, oppure concatenazione di un layer intermedio (~2/3 della profondità) e dell'ultimo	gli strati intermedi spesso rappresentano meglio "dove sta l'informazione"; si sceglie in H2
Proiezione	d → 256	riduce i costi dell'aggregatore
2 layer, 4 teste, attenzione piena		le frasi comunicano fra loro e con `q`: evidenza sparsa (00a §3)
Parametri	~1,5–3M	trascurabili

2. Fase 1 — backbone congelato (consigliata come primo passo)

1. **Estrazione degli stati**: un forward del modello del verdetto su tutto `ev_head` (02e); si salvano su disco solo `q` e `h_k` (in bf16).
   - Spazio: 20k righe × ~60 frasi × 3.840 valori × 2 byte ≈ 9 GB per un layer. Gestibile; per due layer il doppio.
   - Tempo: un'inferenza sul mix, ~1–3 h su L40S.
2. **Training della testa sugli stati salvati**: niente LLM in memoria, si addestra anche sul portatile.

Iperparametro	Valore
Learning rate	1e-3, warmup 5%, cosine
Epoche	5–10 (pochi parametri, dati riutilizzati) con early stopping su `dev_evid_select`
Batch	64 righe
Loss	`L_evidenza` di 04 (BCE pesata + γ·ranking)
Regolarizzazione	dropout 0,1; weight decay 0,01

Vantaggi: il verdetto non cambia (le lettere non vengono toccate); esperimenti su layer, aggregatore e loss in minuti, non ore.

3. Fase 2 — training congiunto (solo se la fase 1 non basta)

Quando: F1 o recall dell'insieme completo sotto gli obiettivi (00_overview.md §5), soprattutto su `test_evid_sparse`-like (righe sparse di `dev_evid_select`).

Parametro	Valore
Inizializzazione	adapter LoRA del verdetto + testa della fase 1
Moduli LoRA	gli stessi del piano binario
Learning rate	LoRA 5e-6 – 1e-5 (basso: il verdetto è già addestrato); testa 2e-4
Loss	`L_verdetto + λ · L_evidenza`, $\lambda = 0{,}3$
Dati	`ev_joint` (tutte le righe; evidenza mascherata dove manca)
Epoche	1
Durata	come una corsa del piano binario + ~10%

Implementazione: nel forward del modello con LoRA si prendono gli stati nascosti finali (`output_hidden_states` o un hook sul layer scelto) nelle posizioni `answer_pos` e `sent_end_pos`; le lettere si calcolano solo in `answer_pos` con le due righe della testa del vocabolario (come nel piano binario); la testa di evidenza riceve `q` e `h`.

4. Cosa controllare

Metrica	Su cosa	Allarme
F1 per frase, recall dell'insieme completo	`dev_evid_select`	non sale sopra E-LEX (01)
F1 per fascia di distanza	righe sparse di `dev_evid_select`	crollo oltre 10 frasi → aggregatore o dati sparsi insufficienti
Numero medio di frasi evidenziate	`dev_evid_select`	molto sopra o sotto il numero gold → `τ_e` o `α` da rivedere
Verdetto (solo fase 2)	`dev_select`	BAcc −0,3 o ECE +0,01 rispetto al modello del piano binario

5. Scelta della soglia di evidenza `τ_e`

Su `dev_evid_select`, fra le righe supportate giudicate supportate: si sceglie `τ_e` che massimizza l'F1 per frase, con un vincolo pratico (al massimo 5 frasi mostrate; almeno 1 se il verdetto è "supportato": se nessuna supera `τ_e`, si mostra la frase con punteggio più alto). Conferma su `dev_evid_confirm`.

6. Test prima del training

Test	Cosa verifica
Fase 1: stati estratti = stati del forward in produzione	stessa funzione di rendering, stesso modello, differenza $< 1e\text{-}3$
Overfit di 64 righe	la testa arriva a F1 ≈ 1 sul training
Fase 2: passo zero	prima di aggiornare, verdetto identico al modello del piano binario

Criteri per passare allo step successivo

- Testa che supera E-LEX ed E-ATT su `dev_evid_select` con margine, confermata su `dev_evid_confirm`.
- Verdetto invariato (fase 1) o entro i limiti (fase 2).